

Refatoração de Testes e Detecção de Test Smells

João Augusto Aquino - 803250

Testes de Software

1. Análise de Test Smells

Durante a análise do arquivo `userService.smelly.test.js`, foram identificados diversos problemas de qualidade que comprometem a legibilidade, manutenção e confiabilidade da suíte de testes. Os três principais Test Smells encontrados foram:

1.1 Lógica Condicional em Testes (Conditional Logic Test)

O teste "deve desativar usuários se eles não forem administradores" utilizava um laço `for` juntamente com uma estrutura condicional `if/else` para validar diferentes cenários em um único teste.

```
for (const user of todosOsUsuarios) {  
  const resultado = userService.deactivateUser(user.id);  
  
  if (!user.isAdmin) {  
    expect(resultado).toBe(true);  
  } else {  
    expect(resultado).toBe(false);  
  }  
}
```

Esse tipo de abordagem é considerado um Test Smell porque um teste deve validar apenas um comportamento específico. A presença de estruturas condicionais faz com que diferentes caminhos de execução sejam percorridos, dificultando a identificação da causa de falhas.

O principal risco é a redução da clareza do teste. Quando ocorre uma falha, torna-se mais difícil determinar qual cenário específico apresentou problema.

1.2 Teste Frágil (Fragile Test)

O teste responsável pela geração do relatório verificava exatamente a formatação produzida pelo método `generateUserReport()`.

```
const linhaEsperada =  
`ID: ${usuario1.id}, Nome: Alice, Status: ativo\n`;  
  
expect(relatorio).toContain(linhaEsperada);
```

Esse comportamento caracteriza um teste frágil, pois qualquer alteração na formatação do relatório pode fazer o teste falhar, mesmo que a funcionalidade continue correta.

O risco desse smell é a geração de falsos positivos, fazendo com que modificações simples de apresentação sejam interpretadas como defeitos no sistema.

1.3 Tratamento Inadequado de Exceções

O teste de validação de idade utilizava um bloco try/catch.

```
try {
  userService.createUser('Menor', 'menor@email.com', 17);
} catch (e) {
  expect(e.message).toBe(
    'O usuário deve ser maior de idade.'
  );
}
```

Esse padrão é considerado inadequado porque, caso nenhuma exceção seja lançada, o teste pode ser executado sem realizar nenhuma verificação efetiva.

O risco é permitir que defeitos passem despercebidos, reduzindo a confiabilidade da suíte de testes.

2. Processo de Refatoração

O teste considerado mais problemático foi o responsável pela desativação de usuários.

Antes

```
test('deve desativar usuários se eles não forem administradores', () => {
  const usuarioComum = userService.createUser(
    'Comum',
    'comum@teste.com',
    30
  );

  const usuarioAdmin = userService.createUser(
    'Admin',
    'admin@teste.com',
    40,
    true
  );

  const todosOsUsuarios = [
    usuarioComum,
    usuarioAdmin
  ];

  for (const user of todosOsUsuarios) {
    const resultado =
      userService.deactivateUser(user.id);

    if (!user.isAdmin) {
```

```
    expect(resultado).toBe(true);
  } else {
    expect(resultado).toBe(false);
  }
}
});
```

Depois

```
test('deve desativar usuário comum', () => {
  const usuario =
    userService.createUser(
      'Comum',
      'comum@teste.com',
      30
    );

  const resultado =
    userService.deactivateUser(usuario.id);

  expect(resultado).toBe(true);
});

test('não deve desativar administrador', () => {
  const admin =
    userService.createUser(
      'Admin',
      'admin@teste.com',
      40,
      true
    );

  const resultado =
    userService.deactivateUser(admin.id);

  expect(resultado).toBe(false);
});
```

A refatoração eliminou completamente a lógica condicional e dividiu o comportamento em dois testes independentes. Além disso, foi aplicado o padrão Arrange, Act e Assert (AAA), aumentando a legibilidade e facilitando a manutenção.

Também foram realizadas outras melhorias, como a substituição do bloco try/catch pelo método `toThrow()` do Jest e a redução da dependência de formatação exata no teste de geração de relatório.

3. Relatório da Ferramenta

Foi utilizado o ESLint juntamente com o plugin Jest para realizar a detecção automática de problemas na suíte de testes.

Na primeira execução da ferramenta foram encontrados os seguintes problemas:

- jest/no-conditional-expect
- jest/no-disabled-tests
- jest/expect-expect

Esses avisos e erros confirmaram os problemas identificados manualmente durante a análise inicial.

Após a refatoração dos testes e a criação do arquivo `userService.clean.test.js`, o ESLint deixou de apontar problemas no novo arquivo, demonstrando a melhoria da qualidade da suíte de testes.

```
Found 0 vulnerabilities
PS G:\test-smelly> npx eslint .

G:\test-smelly\test\userService.smelly.test.js
44:9  error  Avoid calling `expect` conditionally`  jest/no-conditional-expect
46:9  error  Avoid calling `expect` conditionally`  jest/no-conditional-expect
49:9  error  Avoid calling `expect` conditionally`  jest/no-conditional-expect
73:7  error  Avoid calling `expect` conditionally`  jest/no-conditional-expect
77:3  warning Tests should not be skipped           jest/no-disabled-tests
77:3  warning Test has no assertions          jest/expect-expect

X6 problems (4 errors, 2 warnings)

PS G:\test-smelly> npx eslint test/userService.clean.test.js
PS G:\test-smelly> npx eslint test/userService.clean.test.js; echo $LASTEXITCODE
0
PS G:\test-smelly>
```

4. Conclusão

A atividade demonstrou que uma suíte de testes pode apresentar problemas de qualidade mesmo quando todos os testes estão passando. A identificação dos Test Smells permitiu compreender como determinadas práticas reduzem a clareza, a manutenibilidade e a confiabilidade dos testes.

A utilização do ESLint automatizou a detecção desses problemas, servindo como uma ferramenta de apoio ao desenvolvedor. Já a aplicação de boas práticas, como o padrão Arrange, Act e Assert e a separação de responsabilidades entre testes, resultou em uma suíte mais legível, robusta e preparada para futuras modificações no sistema.

Dessa forma, conclui-se que testes limpos e ferramentas de análise estática contribuem diretamente para a qualidade e sustentabilidade de projetos de software.

